Cyclic redundancy check

A **cyclic redundancy check** (**CRC**) is an error-detecting code commonly used in digital networks and storage devices to detect accidental changes to digital data. Blocks of data entering these systems get a short *check value* attached, based on the remainder of a polynomial division of their contents. On retrieval, the calculation is repeated and, in the event the check values do not match, corrective action can be taken against data corruption. CRCs can be used for error correction (see bitfilters).[1]

CRCs are so called because the *check* (data verification) value is a *redundancy* (it expands the message without adding information) and the algorithm is based on *cyclic* codes. CRCs are popular because they are simple to implement in binary hardware, easy to analyze mathematically, and particularly good at detecting common errors caused by noise in transmission channels. Because the check value has a fixed length, the function that generates it is occasionally used as a hash function.

Introduction

CRCs are based on the theory of cyclic error-correcting codes. The use of systematic cyclic codes, which encode messages by adding a fixed-length check value, for the purpose of error detection in communication networks, was first proposed by W. Wesley Peterson in 1961.[2] Cyclic codes are not only simple to implement but have the benefit of being particularly well suited for the detection of burst errors: contiguous sequences of erroneous data symbols in messages. This is important because burst errors are common transmission errors in many communication channels, including magnetic and optical storage devices. Typically an n-bit CRC applied to a data block of arbitrary length will detect any single error burst not longer than n bits, and the fraction of all longer error bursts that it will detect is approximately $(1 - 2^{-n})$.

Specification of a CRC code requires definition of a so-called generator polynomial. This polynomial becomes the divisor in a polynomial long division, which takes the message as the dividend and in which the quotient is discarded and the remainder becomes the result. The important caveat is that the polynomial coefficients are calculated according to the arithmetic of a finite field, so the addition operation can always be performed bitwise-parallel (there is no carry between digits).

In practice, all commonly used CRCs employ the finite field of two elements, GF(2). The two elements are usually called 0 and 1, comfortably matching computer architecture.

A CRC is called an n-bit CRC when its check value is n bits long. For a given n, multiple CRCs are possible, each with a different polynomial. Such a polynomial has highest degree n, which means it has $n + 1$ terms. In other words, the polynomial has a length of $n + 1$; its encoding requires $n + 1$ bits. Note that most polynomial specifications either drop the MSb or LSb, since they are always 1. The CRC and associated polynomial typically have a name of the form CRC-n-XXX as in the table below.

The simplest error-detection system, the parity bit, is in fact a 1-bit CRC: it uses the generator polynomial $x + 1$ (two terms),[3] and has the name CRC-1.

Application

A CRC-enabled device calculates a short, fixed-length binary sequence, known as the *check value* or *CRC*, for each block of data to be sent or stored and appends it to the data, forming a *codeword*.

When a codeword is received or read, the device either compares its check value with one freshly calculated from the data block, or equivalently, performs a CRC on the whole codeword and compares the resulting check value with an expected *residue* constant.

If the CRC values do not match, then the block contains a data error.

The device may take corrective action, such as rereading the block or requesting that it be sent again. Otherwise, the data is assumed to be error-free (though, with some small probability, it may contain undetected errors; this is inherent in the nature of error-checking).[4]

Data integrity

CRCs are specifically designed to protect against common types of errors on communication channels, where they can provide quick and reasonable assurance of the integrity of messages delivered. However, they are not suitable for protecting against intentional alteration of data.

Firstly, as there is no authentication, an attacker can edit a message and recompute the CRC without the substitution being detected. When stored alongside the data, CRCs and cryptographic hash functions by themselves do not protect against *intentional* modification of data. Any application that requires protection against such attacks must use cryptographic authentication mechanisms, such as message authentication codes or digital signatures (which are commonly based on cryptographic hash functions).

Secondly, unlike cryptographic hash functions, CRC is an easily reversible function, which makes it unsuitable for use in digital signatures.[5]

Thirdly, CRC satisfies a relation similar to that of a linear function (or more accurately, an affine function):[6]

$$\mathrm{CRC}(x \oplus y) = \mathrm{CRC}(x) \oplus \mathrm{CRC}(y) \oplus c$$

where c depends on the length of x and y. This can be also stated as follows, where x, y and z have the same length

$$\mathrm{CRC}(x \oplus y \oplus z) = \mathrm{CRC}(x) \oplus \mathrm{CRC}(y) \oplus \mathrm{CRC}(z);$$

as a result, even if the CRC is encrypted with a stream cipher that uses XOR as its combining operation (or mode of block cipher which effectively turns it into a stream cipher, such as OFB or CFB), both the message and the associated CRC can be manipulated without knowledge of the encryption key; this was one of the well-known design flaws of the Wired Equivalent Privacy (WEP) protocol.[7]

Computation

To compute an n-bit binary CRC, line the bits representing the input in a row, and position the (n + 1)-bit pattern representing the CRC's divisor (called a "polynomial") underneath the left end of the row.

In this example, we shall encode 14 bits of message with a 3-bit CRC, with a polynomial $x^3 + x + 1$. The polynomial is written in binary as the coefficients; a 3rd-degree polynomial has 4 coefficients ($1x^3 + 0x^2 + 1x + 1$). In this case, the coefficients are 1, 0, 1 and 1. The result of the calculation is 3 bits long, which is why it is called a 3-bit CRC. However, 4 bits are needed to explicitly state the polynomial.

Start with the message to be encoded:

```
11010011101100
```

This is first padded with zeros corresponding to the bit length n of the CRC. This is done so that the resulting code word is in systematic form. Here is the first calculation for computing a 3-bit CRC:

```
11010011101100 000 <--- input padded by 3 bits from the right
1011               <--- divisor (4 bits) = x^3 + x + 1
------------------
01100011101100 000 <--- result
```

The algorithm acts on the bits directly above the divisor in each step. The result for that iteration is the bitwise XOR of the polynomial divisor with the bits above it. The bits not above the divisor are simply copied directly below for that step. The divisor is then shifted right to align with the highest remaining 1 bit in the input, and the process is repeated until the divisor reaches the right-hand end of the input row. Here is the entire calculation:

```
11010011101100 000 <--- input padded by 3 bits from the right
1011               <--- divisor
01100011101100 000 <--- result (the first four bits are the XOR with the divisor beneath, the rest of
the bits are unchanged)
 1011              <--- divisor ...
00111011101100 000
  1011
00010111101100 000
   1011
00000001101100 000 <--- the divisor moves over to align with the next 1 in the dividend (since
quotient for that step was zero)
       1011             (in other words, it doesn't necessarily move one bit per iteration)
00000000110100 000
        1011
00000000011000 000
         1011
00000000001110 000
          1011
00000000000101 000
           101 1
-----------------
00000000000000 100 <--- remainder (3 bits).  Division algorithm stops here as dividend is equal to
zero.
```

Since the leftmost divisor bit zeroed every input bit it touched, when this process ends the only bits in the input row that can be nonzero are the n bits at the right-hand end of the row. These n bits are the remainder of the division step, and will also be the value of the CRC function (unless the chosen CRC

specification calls for some postprocessing).

The validity of a received message can easily be verified by performing the above calculation again, this time with the check value added instead of zeroes. The remainder should equal zero if there are no detectable errors.

```
11010011101100 100 <--- input with check value
1011               <--- divisor
01100011101100 100 <--- result
 1011              <--- divisor ...
00111011101100 100

......

00000000001110 100
          1011
00000000000101 100
           101 1
------------------
00000000000000 000 <--- remainder
```

The following Python code outlines a function which will return the initial CRC remainder for a chosen input and polynomial, with either 1 or 0 as the initial padding. This code works with string inputs rather than raw numbers:

```python
def crc_remainder(input_bitstring, polynomial_bitstring, initial_filler):
    """Calculate the CRC remainder of a string of bits using a chosen polynomial.
    initial_filler should be '1' or '0'.
    """
    polynomial_bitstring = polynomial_bitstring.lstrip("0")
    len_input = len(input_bitstring)
    initial_padding = (len(polynomial_bitstring) - 1) * initial_filler
    input_padded_array = list(input_bitstring + initial_padding)
    while "1" in input_padded_array[:len_input]:
        cur_shift = input_padded_array.index("1")
        for i in range(len(polynomial_bitstring)):
            input_padded_array[cur_shift + i] \
            = str(int(polynomial_bitstring[i] != input_padded_array[cur_shift + i]))
    return "".join(input_padded_array)[len_input:]

def crc_check(input_bitstring, polynomial_bitstring, check_value):
    """Calculate the CRC check of a string of bits using a chosen polynomial."""
    polynomial_bitstring = polynomial_bitstring.lstrip("0")
    len_input = len(input_bitstring)
    initial_padding = check_value
    input_padded_array = list(input_bitstring + initial_padding)
    while "1" in input_padded_array[:len_input]:
        cur_shift = input_padded_array.index("1")
        for i in range(len(polynomial_bitstring)):
            input_padded_array[cur_shift + i] \
            = str(int(polynomial_bitstring[i] != input_padded_array[cur_shift + i]))
    return ("1" not in "".join(input_padded_array)[len_input:])
```

```python
>>> crc_remainder('11010011101100', '1011', '0')
'100'
>>> crc_check('11010011101100', '1011', '100')
True
```

Mathematics

Mathematical analysis of this division-like process reveals how to select a divisor that guarantees good error-detection properties. In this analysis, the digits of the bit strings are taken as the coefficients of a polynomial in some variable *x*—coefficients that are elements of the finite field GF(2) (the integers modulo 2, i.e. either a zero or a one), instead of more familiar numbers. The set of binary polynomials is a mathematical ring.

Designing polynomials

The selection of the generator polynomial is the most important part of implementing the CRC algorithm. The polynomial must be chosen to maximize the error-detecting capabilities while minimizing overall collision probabilities.

The most important attribute of the polynomial is its length (largest degree(exponent) +1 of any one term in the polynomial), because of its direct influence on the length of the computed check value.

The most commonly used polynomial lengths are 9 bits (CRC-8), 17 bits (CRC-16), 33 bits (CRC-32), and 65 bits (CRC-64).[3]

A CRC is called an *n*-bit CRC when its check value is *n*-bits. For a given *n*, multiple CRCs are possible, each with a different polynomial. Such a polynomial has highest degree *n*, and hence *n* + 1 terms (the polynomial has a length of *n* + 1). The remainder has length *n*. The CRC has a name of the form CRC-*n*-XXX.

The design of the CRC polynomial depends on the maximum total length of the block to be protected (data + CRC bits), the desired error protection features, and the type of resources for implementing the CRC, as well as the desired performance. A common misconception is that the "best" CRC polynomials are derived from either irreducible polynomials or irreducible polynomials times the factor 1 + *x*, which adds to the code the ability to detect all errors affecting an odd number of bits.[8] In reality, all the factors described above should enter into the selection of the polynomial and may lead to a reducible polynomial. However, choosing a reducible polynomial will result in a certain proportion of missed errors, due to the quotient ring having zero divisors.

The advantage of choosing a primitive polynomial as the generator for a CRC code is that the resulting code has maximal total block length in the sense that all 1-bit errors within that block length have different remainders (also called syndromes) and therefore, since the remainder is a linear function of the block, the code can detect all 2-bit errors within that block length. If r is the degree of the primitive generator polynomial, then the maximal total block length is $2^r - 1$, and the associated code is able to detect any single-bit or double-bit errors.[9] However, if we use the generator polynomial $g(x) = p(x)(1 + x)$, where p is a primitive polynomial of degree $r - 1$, then the maximal total block length is $2^{r-1} - 1$, and the code is able to detect single, double, triple and any odd number of errors.

A polynomial $g(x)$ that admits other factorizations may be chosen then so as to balance the maximal total blocklength with a desired error detection power. The BCH codes are a powerful class of such polynomials. They subsume the two examples above. Regardless of the reducibility properties of a

generator polynomial of degree *r*, if it includes the "+1" term, the code will be able to detect error patterns that are confined to a window of *r* contiguous bits. These patterns are called "error bursts".

Specification

The concept of the CRC as an error-detecting code gets complicated when an implementer or standards committee uses it to design a practical system. Here are some of the complications:

- Sometimes an implementation **prefixes a fixed bit pattern** to the bitstream to be checked. This is useful when clocking errors might insert 0-bits in front of a message, an alteration that would otherwise leave the check value unchanged.
- Usually, but not always, an implementation **appends *n* 0-bits** (*n* being the size of the CRC) to the bitstream to be checked before the polynomial division occurs. Such appending is explicitly demonstrated in the Computation of CRC article. This has the convenience that the remainder of the original bitstream with the check value appended is exactly zero, so the CRC can be checked simply by performing the polynomial division on the received bitstream and comparing the remainder with zero. Due to the associative and commutative properties of the exclusive-or operation, practical table driven implementations can obtain a result numerically equivalent to zero-appending without explicitly appending any zeroes, by using an equivalent,[8] faster algorithm that combines the message bitstream with the stream being shifted out of the CRC register.
- Sometimes an implementation **exclusive-ORs a fixed bit pattern** into the remainder of the polynomial division.
- **Bit order:** Some schemes view the low-order bit of each byte as "first", which then during polynomial division means "leftmost", which is contrary to our customary understanding of "low-order". This convention makes sense when serial-port transmissions are CRC-checked in hardware, because some widespread serial-port transmission conventions transmit bytes least-significant bit first.
- **Byte order**: With multi-byte CRCs, there can be confusion over whether the byte transmitted first (or stored in the lowest-addressed byte of memory) is the least-significant byte (LSB) or the most-significant byte (MSB). For example, some 16-bit CRC schemes swap the bytes of the check value.
- **Omission of the high-order bit** of the divisor polynomial: Since the high-order bit is always 1, and since an *n*-bit CRC must be defined by an (*n* + 1)-bit divisor which overflows an *n*-bit register, some writers assume that it is unnecessary to mention the divisor's high-order bit.
- **Omission of the low-order bit** of the divisor polynomial: Since the low-order bit is always 1, authors such as Philip Koopman represent polynomials with their high-order bit intact, but without the low-order bit (the x^0 or 1 term). This convention encodes the polynomial complete with its degree in one integer.

These complications mean that there are three common ways to express a polynomial as an integer: the first two, which are mirror images in binary, are the constants found in code; the third is the number found in Koopman's papers. *In each case, one term is omitted.* So the polynomial $x^4 + x + 1$ may be transcribed as:

- 0x3 = 0b0011, representing $x^4 + (0x^3 + 0x^2 + 1x^1 + 1x^0)$ (MSB-first code)
- 0xC = 0b1100, representing $(1x^0 + 1x^1 + 0x^2 + 0x^3) + x^4$ (LSB-first code)
- 0x9 = 0b1001, representing $(1x^4 + 0x^3 + 0x^2 + 1x^1) + x^0$ (Koopman notation)

In the table below they are shown as:

Examples of CRC representations

Name	Normal	Reversed	Reversed reciprocal
CRC-4	0x3	0xC	0x9

Obfuscation

CRCs in proprietary protocols might be obfuscated by using a non-trivial initial value and a final XOR, but these techniques do not introduce cryptographic strength into the algorithm and can be reverse engineered using straightforward methods.[10]

Standards and common use

Numerous varieties of cyclic redundancy checks have been incorporated into technical standards. By no means does one algorithm, or one of each degree, suit every purpose; Koopman and Chakravarty recommend selecting a polynomial according to the application requirements and the expected distribution of message lengths.[11] The number of distinct CRCs in use has confused developers, a situation which authors have sought to address.[8] There are three polynomials reported for CRC-12,[11] twenty-two conflicting definitions of CRC-16, and seven of CRC-32.[12]

The polynomials commonly applied are not the most efficient ones possible. Since 1993, Koopman, Castagnoli and others have surveyed the space of polynomials between 3 and 64 bits in size,[11][13][14][15] finding examples that have much better performance (in terms of Hamming distance for a given message size) than the polynomials of earlier protocols, and publishing the best of these with the aim of improving the error detection capacity of future standards.[14] In particular, iSCSI and SCTP have adopted one of the findings of this research, the CRC-32C (Castagnoli) polynomial.

The design of the 32-bit polynomial most commonly used by standards bodies, CRC-32-IEEE, was the result of a joint effort for the Rome Laboratory and the Air Force Electronic Systems Division by Joseph Hammond, James Brown and Shyan-Shiang Liu of the Georgia Institute of Technology and Kenneth Brayer of the Mitre Corporation. The earliest known appearances of the 32-bit polynomial were in their 1975 publications: Technical Report 2956 by Brayer for Mitre, published in January and released for public dissemination through DTIC in August,[16] and Hammond, Brown and Liu's report for the Rome Laboratory, published in May.[17] Both reports contained contributions from the other team. During December 1975, Brayer and Hammond presented their work in a paper at the IEEE National Telecommunications Conference: the IEEE CRC-32 polynomial is the generating polynomial of a Hamming code and was selected for its error detection performance.[18] Even so, the Castagnoli CRC-32C polynomial used in iSCSI or SCTP matches its performance on messages from 58 bits to 131 kbits, and outperforms it in several size ranges including the two most common sizes of Internet packet.[14] The ITU-T G.hn standard also uses CRC-32C to detect errors in the payload (although it uses CRC-16-CCITT for PHY headers).

CRC-32C computation is implemented in hardware as an operation (`CRC32`) of SSE4.2 instruction set, first introduced in Intel processors' Nehalem microarchitecture. ARM AArch64 architecture also provides hardware acceleration for both CRC-32 and CRC-32C operations.

Polynomial representations

The table below lists only the polynomials of the various algorithms in use. Variations of a particular protocol can impose pre-inversion, post-inversion and reversed bit ordering as described above. For example, the CRC-32 used in Gzip and Bzip2 use the same polynomial, but Gzip employs reversed bit ordering, while Bzip2 does not.[12] Note that even parity polynomials in GF(2) with degree greater than 1 are never primitive. Even parity polynomial marked as primitive in this table represent a primitive polynomial multiplied by $(x+1)$. The most significant bit of a polynomial is always 1, and is not shown in the hex representations.

Name	Uses	Polynomial representations		
		Normal	Reversed	Reciprocal
CRC-1	most hardware; also known as *parity bit*	0x1	0x1	0x1
		$x + 1$		
CRC-3-GSM	mobile networks[23]	0x3	0x6	0x5
		$x^3 + x + 1$		
CRC-4-ITU	ITU-T G.704 (http://www.itu.int/rec/T-REC-G.704-199810-I/en), p. 12	0x3	0xC	0x9
		$x^4 + x + 1$		
CRC-5-EPC	Gen 2 RFID[25]	0x09	0x12	0x05
		$x^5 + x^3 + 1$		
CRC-5-ITU	ITU-T G.704 (http://www.itu.int/rec/T-REC-G.704-199810-I/en), p. 9	0x15	0x15	0x0B
		$x^5 + x^4 + x^2 + 1$		
CRC-5-USB	USB token packets	0x05	0x14	0x09
		$x^5 + x^2 + 1$		
CRC-6-CDMA2000-A	mobile networks[26]	0x27	0x39	0x33
CRC-6-CDMA2000-B	mobile networks[26]	0x07	0x38	0x31
CRC-6-DARC	Data Radio Channel[27]	0x19	0x26	0x0D
CRC-6-GSM	mobile networks[23]	0x2F	0x3D	0x3B
		$x^6 + x^5 + x^3 + x^2 + x + 1$		
CRC-6-ITU	ITU-T G.704 (http://www.itu.int/rec/T-REC-G.704-199810-I/en), p. 3	0x03	0x30	0x21
		$x^6 + x + 1$		
CRC-7	telecom systems, ITU-T G.707 (http://www.itu.int/rec/T-REC-G.707/en), ITU-T G.832 (http://www.itu.int/rec/T-REC-G.832/en), MMC, SD	0x09	0x48	0x11
		$x^7 + x^3 + 1$		
CRC-7-MVB	Train Communication Network, IEC 60870-5[29]	0x65	0x53	0x27

CRC-8	DVB-S2[30]	0xD5	0xAB	0x57
		$x^8 + x^7 + x^6 + x^4 + x^2 + 1$		
CRC-8-AUTOSAR	automotive integration,[32] OpenSafety[33]	0x2F	0xF4	0xE9
		$x^8 + x^5 + x^3 + x^2 + x + 1$		
CRC-8-Bluetooth	wireless connectivity[34]	0xA7	0xE5	0xCB
		$x^8 + x^7 + x^5 + x^2 + x + 1$		
CRC-8-CCITT	ITU-T I.432.1 (02/99) (http://www.itu.int/rec/T-REC-I.432.1-199902-I/en); ATM HEC, ISDN HEC and cell delineation, SMBus PEC	0x07	0xE0	0xC1
		$x^8 + x^2 + x + 1$		
CRC-8-Dallas/Maxim	1-Wire bus[35]	0x31	0x8C	0x19
		$x^8 + x^5 + x^4 + 1$		
CRC-8-DARC	Data Radio Channel[27]	0x39	0x9C	0x39
		$x^8 + x^5 + x^4 + x^3 + 1$		
CRC-8-GSM-B	mobile networks[23]	0x49	0x92	0x25
		$x^8 + x^6 + x^3 + 1$		
CRC-8-SAE J1850	AES3; OBD	0x1D	0xB8	0x71
		$x^8 + x^4 + x^3 + x^2 + 1$		
CRC-8-WCDMA	mobile networks[26][36]	0x9B	0xD9	0xB3
		$x^8 + x^7 + x^4 + x^3 + x + 1$		
CRC-10	ATM; ITU-T I.610 (http://www.itu.int/rec/T-REC-I.610/en)	0x233	0x331	0x263
		$x^{10} + x^9 + x^5 + x^4 + x + 1$		
CRC-10-CDMA2000	mobile networks[26]	0x3D9	0x26F	0x0DF
CRC-10-GSM	mobile networks[23]	0x175	0x2BA	0x175
CRC-11	FlexRay[37]	0x385	0x50E	0x21D
		$x^{11} + x^9 + x^8 + x^7 + x^2 + 1$		
CRC-12	telecom systems[38][39]	0x80F	0xF01	0xE03
		$x^{12} + x^{11} + x^3 + x^2 + x + 1$		
CRC-12-CDMA2000	mobile networks[26]	0xF13	0xC8F	0x91F
CRC-12-GSM	mobile networks[23]	0xD31	0x8CB	0x197
CRC-13-BBC	Time signal, Radio	0x1CF5	0x15E7	0x0BCF

	teleswitch[40][41]	$x^{13}+x^{12}+x^{11}+x^{10}+x^{7}+x^{6}+x^{5}+x^{4}+x^{2}+1$		
CRC-14-DARC	Data Radio Channel[27]	0x0805	0x2804	0x1009
CRC-14-GSM	mobile networks[23]	0x202D	0x2D01	0x1A03
CRC-15-CAN		0xC599[42][43]	0x4CD1	0x19A3
		$x^{15}+x^{14}+x^{10}+x^{8}+x^{7}+x^{4}+x^{3}+1$		
CRC-15-MPT1327	[44]	0x6815	0x540B	0x2817
CRC-16-Chakravarty	Optimal for payloads ≤64 bits[29]	0x2F15	0xA8F4	0x51E9
CRC-16-ARINC	ACARS applications[45]	0xA02B	0xD405	0xA80B
CRC-16-CCITT	X.25, V.41, HDLC *FCS*, XMODEM, Bluetooth, PACTOR, SD, DigRF, many others; known as *CRC-CCITT*	0x1021	0x8408	0x811
		$x^{16}+x^{12}+x^{5}+1$		
CRC-16-CDMA2000	mobile networks[26]	0xC867	0xE613	0xCC27
CRC-16-DECT	cordless telephones[46]	0x0589	0x91A0	0x2341
		$x^{16}+x^{10}+x^{8}+x^{7}+x^{3}+1$		
CRC-16-T10-DIF	SCSI DIF	0x8BB7[47]	0xEDD1	0xDBA3
		$x^{16}+x^{15}+x^{11}+x^{9}+x^{8}+x^{7}+x^{5}+x^{4}+x^{2}+x+1$		
CRC-16-DNP	DNP, IEC 870, M-Bus	0x3D65	0xA6BC	0x4D79
		$x^{16}+x^{13}+x^{12}+x^{11}+x^{10}+x^{8}+x^{6}+x^{5}+x^{2}+1$		
CRC-16-IBM	Bisync, Modbus, USB, ANSI X3.28 (https://web.archive.org/web/20091001172850/http://www.incits.org/press/1997/pr97020.htm), SIA DC-07, many others; also known as *CRC-16* and *CRC-16-ANSI*	0x8005	0xA001	0x4003
		$x^{16}+x^{15}+x^{2}+1$		
CRC-16-OpenSafety-A	safety fieldbus[33]	0x5935	0xAC9A	0x5935
CRC-16-OpenSafety-B	safety fieldbus[33]	0x755B	0xDAAE	0xB55D

CRC-16-Profibus	fieldbus networks[48]	0x1DCF	0xF3B8	0xE771
Fletcher-16	Used in Adler-32 A & B Checksums	Often confused to be a CRC, but actually a checksum; see Fletcher's checksu		
CRC-17-CAN	CAN FD[49]	0x1685B	0x1B42D	0x1685B
CRC-21-CAN	CAN FD[49]	0x102899	0x132281	0x064503
CRC-24	FlexRay[37]	0x5D6DCB	0xD3B6BA	0xA76D75
		$x^{24}+x^{22}+x^{20}+x^{19}+x^{18}+x^{16}+x^{14}+x^{13}+x^{11}+x^{10}+x^{8}+x^{7}+x^{6}$ -		
CRC-24-Radix-64	OpenPGP, RTCM104v3	0x864CFB	0xDF3261	0xBE64C3
		$x^{24}+x^{23}+x^{18}+x^{17}+x^{14}+x^{11}+x^{10}+x^{7}+x^{6}+x^{5}+x^{4}+x^{3}+x+1$		
CRC-24-WCDMA	Used in OS-9 RTOS. Residue = 0x800FE3.[50]	0x800063	0xC60001	0x8C0003
		$x^{24}+x^{23}+x^{6}+x^{5}+x+1$		
CRC-30	CDMA	0x2030B9C7	0x38E74301	0x31CE8603
		$x^{30}+x^{29}+x^{21}+x^{20}+x^{15}+x^{13}+x^{12}+x^{11}+x^{8}+x^{7}+x^{6}+x^{2}+x+1$		
CRC-32	ISO 3309 (HDLC), ANSI X3.66 (ADCCP), FIPS PUB 71, FED-STD-1003, ITU-T V.42, ISO/IEC/IEEE 802-3 (Ethernet), ISO/IEC/IEEE 802-11 (Wi-Fi), SATA, NVMe,[52] MPEG-2, PKZIP, Gzip, Bzip2, PCI Express, HDMI, POSIX cksum,[53] PNG,[54] ZMODEM, many others	0x04C11DB7	0xEDB88320	0xDB710641
		$x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^{8}+x^{7}+x^{5}+x^{4}+x^{2}+$		
CRC-32C (Castagnoli)	iSCSI, SCTP, G.hn payload, SSE4.2, Btrfs, ext4, ReFS,[55] VHDX,[56] Ceph	0x1EDC6F41	0x82F63B78	0x05EC76F1
		$x^{32}+x^{28}+x^{27}+x^{26}+x^{25}+x^{23}+x^{22}+x^{20}+x^{19}+x^{18}+x^{14}+x^{13}+x^{1}$		
CRC-32K (Koopman {1,3,28})	Excellent at Ethernet frame length, poor performance with long files	0x741B8CD7	0xEB31D82E	0xD663B05D
		$x^{32}+x^{30}+x^{29}+x^{28}+x^{26}+x^{20}+x^{19}+x^{17}+x^{16}+x^{15}+x^{11}+x^{10}+x^{7}$		
CRC-32K$_2$ (Koopman {1,1,30})	Excellent at Ethernet frame length, poor performance with long files	0x32583499	0x992C1A4C	0x32583499

CRC-32Q	aviation; AIXM[57]	0x814141AB	0xD5828281	0xAB050503
		$x^{32}+x^{31}+x^{24}+x^{22}+x^{16}+x^{14}+x^{8}+x^{7}+x^{5}+x^{3}+x+1$		
Adler-32		Often confused to be a CRC, but actually a checksum; see Adler-32		
CRC-40-GSM	GSM control channel[58][59][60]	0x0004820009	0x9000412000	0x2000824001
		$x^{40}+x^{26}+x^{23}+x^{17}+x^{3}+1=(x^{23}+1)(x^{17}+x^{3}+1)$		
CRC-64-ECMA	ECMA-182 (https://ecma-international.org/publications-and-standards/standards/ecma-182/) p. 51, XZ Utils	0x42F0E1EBA9EA3693	0xC96C5795D7870F42	0x92D8AF2BAF0E1E8
		$x^{64}+x^{62}+x^{57}+x^{55}+x^{54}+x^{53}+x^{52}+x^{47}+x^{46}+x^{45}+x^{40}+x^{39}+x^{3}$ $x^{32}+x^{31}+x^{29}+x^{27}+x^{24}+x^{23}+x^{22}+x^{21}+x^{19}+x^{17}+x^{13}+x^{12}+x^{1}$		
CRC-64-ISO	ISO 3309 (HDLC), Swiss-Prot/TrEMBL; considered weak for hashing[61]	0x000000000000001B	0xD800000000000000	0xB000000000000001
		$x^{64}+x^{4}+x^{3}+x+1$		

Implementations

- Implementation of CRC32 in GNU Radio up to 3.6.1 (ca. 2012) (https://web.archive.org/web/20130715065157/http://gnuradio.org/redmine/projects/gnuradio/repository/revisions/1cb52da49230c64c3719b4ab944ba1cf5a9abb92/entry/gr-digital/lib/digital_crc32.cc)
- C class code for CRC checksum calculation with many different CRCs to choose from (https://sourceforge.net/projects/crccalculator/files/CRC/)
- CRC-32 - Rosetta Code (https://rosettacode.org/wiki/CRC-32)

CRC catalogues

- Catalogue of parametrised CRC algorithms (https://reveng.sourceforge.io/crc-catalogue/all.htm)
- CRC Polynomial Zoo (http://users.ece.cmu.edu/~koopman/crc/crc32.html)

See also

- Checksum
- Computation of cyclic redundancy checks
- Information security
- List of checksum algorithms
- List of hash functions
- LRC
- Mathematics of cyclic redundancy checks
- Simple file verification

References

1. "An Algorithm for Error Correcting Cyclic Redundance Checks" (https://web.archive.org/web/20170720165847/http://www.drdobbs.com/an-algorithm-for-error-correcting-cyclic/184401662). *drdobbs.com*. Archived from the original (http://www.drdobbs.com/an-algorithm-for-error-correcting-cyclic/184401662) on 20 July 2017. Retrieved 28 June 2017.
2. Peterson, W. W.; Brown, D. T. (January 1961). "Cyclic Codes for Error Detection". *Proceedings of the IRE*. **49** (1): 228–235. Bibcode:1961PIRE...49..228P (https://ui.adsabs.harvard.edu/abs/1961PIRE...49..228P). doi:10.1109/JRPROC.1961.287814 (https://doi.org/10.1109%2FJRPROC.1961.287814). S2CID 51666741 (https://api.semanticscholar.org/CorpusID:51666741).
3. Ergen, Mustafa (21 January 2008). "2.3.3 Error Detection Coding". *Mobile Broadband*. Springer. pp. 29–30. doi:10.1007/978-0-387-68192-4_2 (https://doi.org/10.1007%2F978-0-387-68192-4_2). ISBN 978-0-387-68192-4.
4. Ritter, Terry (February 1986). "The Great CRC Mystery" (http://www.ciphersbyritter.com/ARTS/CRCMYST.HTM). *Dr. Dobb's Journal*. **11** (2): 26–34, 76–83. Archived (https://web.archive.org/web/20090416095111/http://www.ciphersbyritter.com/ARTS/CRCMYST.HTM) from the original on 16 April 2009. Retrieved 21 May 2009.
5. Stigge, Martin; Plötz, Henryk; Müller, Wolf; Redlich, Jens-Peter (May 2006). "Reversing CRC – Theory and Practice" (https://web.archive.org/web/20110719042902/http://sar.informatik.hu-berlin.de/research/publications/SAR-PR-2006-05/SAR-PR-2006-05_.pdf) (PDF). Humboldt University Berlin. p. 17. SAR-PR-2006-05. Archived from the original (http://sar.informatik.hu-berlin.de/research/publications/SAR-PR-2006-05/SAR-PR-2006-05_.pdf) (PDF) on 19 July 2011. Retrieved 4 February 2011. "The presented methods offer a very easy and efficient way to modify your data so that it will compute to a CRC you want or at least know in advance."
6. "algorithm design – Why is CRC said to be linear?" (https://crypto.stackexchange.com/a/34013). *Cryptography Stack Exchange*. Retrieved 5 May 2019.
7. Cam-Winget, Nancy; Housley, Russ; Wagner, David; Walker, Jesse (May 2003). "Security Flaws in 802.11 Data Link Protocols" (http://www.cs.berkeley.edu/~daw/papers/wireless-cacm.pdf) (PDF). *Communications of the ACM*. **46** (5): 35–39. CiteSeerX 10.1.1.14.8775 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.14.8775). doi:10.1145/769800.769823 (https://doi.org/10.1145%2F769800.769823). S2CID 3132937 (https://api.semanticscholar.org/CorpusID:3132937). Archived (https://web.archive.org/web/20130526173938/http://www.cs.berkeley.edu/~daw/papers/wireless-cacm.pdf) (PDF) from the original on 26 May 2013. Retrieved 1 November 2017.
8. Williams, Ross N. (24 September 1996). "A Painless Guide to CRC Error Detection Algorithms V3.0" (https://web.archive.org/web/20180402205812/http://www.wolfgang-ehrhardt.de/crc_v3.html). Archived from the original (http://www.wolfgang-ehrhardt.de/crc_v3.html) on 2 April 2018. Retrieved 23 May 2019.
9. Press, WH; Teukolsky, SA; Vetterling, WT; Flannery, BP (2007). "Section 22.4 Cyclic Redundancy and Other Checksums" (http://numerical.recipes/book.html). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.). Cambridge University Press. ISBN 978-0-521-88068-8. Archived (https://web.archive.org/web/20240713014419/http://numerical.recipes/book.html) from the original on 13 July 2024. Retrieved 20 August 2024.
10. Ewing, Gregory C. (March 2010). "Reverse-Engineering a CRC Algorithm" (http://www.cosc.canterbury.ac.nz/greg.ewing/essays/CRC-Reverse-Engineering.html). Christchurch: University of Canterbury. Archived (https://web.archive.org/web/20110807100031/http://www.cosc.canterbury.ac.nz/greg.ewing/essays/CRC-Reverse-Engineering.html) from the original on 7 August 2011. Retrieved 26 July 2011.

11. Koopman, Philip; Chakravarty, Tridib (June 2004). "Cyclic redundancy code (CRC) polynomial selection for embedded networks". *International Conference on Dependable Systems and Networks, 2004* (http://www.ece.cmu.edu/~koopman/roses/dsn04/koopman04_crc_poly_embedded.pdf) (PDF). pp. 145–154. CiteSeerX 10.1.1.648.9080 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.648.9080). doi:10.1109/DSN.2004.1311885 (https://doi.org/10.1109%2FDSN.2004.1311885). ISBN 978-0-7695-2052-0. S2CID 793862 (https://api.semanticscholar.org/CorpusID:793862). Archived (https://web.archive.org/web/20110911095215/http://www.ece.cmu.edu/~koopman/roses/dsn04/koopman04_crc_poly_embedded.pdf) (PDF) from the original on 11 September 2011. Retrieved 14 January 2011.
12. Cook, Greg (15 August 2020). "Catalogue of parametrised CRC algorithms" (https://reveng.sourceforge.io/crc-catalogue/all.htm). Archived (https://web.archive.org/web/20200801122415/https://reveng.sourceforge.io/crc-catalogue/all.htm) from the original on 1 August 2020. Retrieved 18 September 2020.
13. Castagnoli, G.; Bräuer, S.; Herrmann, M. (June 1993). "Optimization of Cyclic Redundancy-Check Codes with 24 and 32 Parity Bits". *IEEE Transactions on Communications*. **41** (6): 883–892. Bibcode:1993ITCom..41..883C (https://ui.adsabs.harvard.edu/abs/1993ITCom..41..883C). doi:10.1109/26.231911 (https://doi.org/10.1109%2F26.231911).
14. Koopman, Philip (July 2002). "32-bit cyclic redundancy codes for Internet applications". *Proceedings International Conference on Dependable Systems and Networks* (http://www.ece.cmu.edu/~koopman/networks/dsn02/dsn02_koopman.pdf) (PDF). pp. 459–468. CiteSeerX 10.1.1.11.8323 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.11.8323). doi:10.1109/DSN.2002.1028931 (https://doi.org/10.1109%2FDSN.2002.1028931). ISBN 978-0-7695-1597-7. S2CID 14775606 (https://api.semanticscholar.org/CorpusID:14775606). Archived (https://web.archive.org/web/20120916185941/http://www.ece.cmu.edu/~koopman/networks/dsn02/dsn02_koopman.pdf) (PDF) from the original on 16 September 2012. Retrieved 14 January 2011.
15. Koopman, Philip (21 January 2016). "Best CRC Polynomials" (https://users.ece.cmu.edu/~koopman/crc/). Carnegie Mellon University. Archived (https://web.archive.org/web/20160120021307/http://users.ece.cmu.edu/~koopman/crc/) from the original on 20 January 2016. Retrieved 26 January 2016.
16. Brayer, Kenneth (August 1975). Evaluation of 32 Degree Polynomials in Error Detection on the SATIN IV Autovon Error Patterns (https://apps.dtic.mil/sti/citations/ADA014825) (Report). National Technical Information Service. ADA014825. Archived (https://web.archive.org/web/20211231093215/https://apps.dtic.mil/sti/citations/ADA014825) from the original on 31 December 2021. Retrieved 31 December 2021.
17. Hammond, Joseph L. Jr.; Brown, James E.; Liu, Shyan-Shiang (1975). "Development of a Transmission Error Model and an Error Control Model" (https://apps.dtic.mil/sti/citations/ADA013939). *NASA Sti/Recon Technical Report N*. **76** (published May 1975): 15344. Bibcode:1975STIN...7615344H (https://ui.adsabs.harvard.edu/abs/1975STIN...7615344H). ADA013939. Archived (https://web.archive.org/web/20211231093212/https://apps.dtic.mil/sti/citations/ADA013939) from the original on 31 December 2021. Retrieved 31 December 2021.
18. Brayer, Kenneth; Hammond, Joseph L. Jr. (December 1975). *Evaluation of error detection polynomial performance on the AUTOVON channel* (https://books.google.com/books?id=2xxGAQAAIAAJ). NTC 75 : National Telecommunications Conference, December 1–3, 1975, New Orleans, Louisiana. Vol. 1. Institute of Electrical and Electronics Engineers. pp. 8–21–5. Bibcode:1975ntc.....1....8B (https://ui.adsabs.harvard.edu/abs/1975ntc.....1....8B). OCLC 32688603 (https://search.worldcat.org/oclc/32688603). 75 CH 1015-7 CSCB.
19. CRCs with even parity detect any odd number of bit errors, at the expense of lower hamming distance for long payloads. Note that parity is computed over the entire generator polynomial, including implied 1 at the beginning or the end. For example, the full representation of CRC-1 is 0x3, which has two 1 bits. Thus, its parity is even.

20. "32 Bit CRC Zoo" (https://users.ece.cmu.edu/~koopman/crc/crc32.html). *users.ece.cmu.edu*. Archived (https://web.archive.org/web/20180319125501/http://users.ece.cmu.edu/~koopman/crc/crc32.html) from the original on 19 March 2018. Retrieved 5 November 2017.
21. Payload means length exclusive of CRC field. A Hamming distance of d means that $d - 1$ bit errors can be detected and $\lfloor (d - 1)/2 \rfloor$ bit errors can be corrected
22. is always achieved for arbitrarily long messages
23. *ETSI TS 100 909* (https://www.etsi.org/deliver/etsi_ts/100900_100999/100909/08.09.00_60/ts_100909v080900p.pdf) (PDF). V8.9.0. Sophia Antipolis, France: European Telecommunications Standards Institute. January 2005. Archived (https://web.archive.org/web/20180417050648/http://www.etsi.org/deliver/etsi_ts/100900_100999/100909/08.09.00_60/ts_100909v080900p.pdf) (PDF) from the original on 17 April 2018. Retrieved 21 October 2016.
24. "3 Bit CRC Zoo" (https://users.ece.cmu.edu/~koopman/crc/crc3.html). *users.ece.cmu.edu*. Archived (https://web.archive.org/web/20180407230324/http://users.ece.cmu.edu/~koopman/crc/crc3.html) from the original on 7 April 2018. Retrieved 19 January 2018.
25. *Class-1 Generation-2 UHF RFID Protocol* (http://www.gs1.org/gsmp/kc/epcglobal/uhfc1g2/uhfc1g2_1_2_0-standard-20080511.pdf) (PDF). 1.2.0. EPCglobal. 23 October 2008. p. 35. Archived (https://web.archive.org/web/20120319154207/http://www.gs1.org/gsmp/kc/epcglobal/uhfc1g2/uhfc1g2_1_2_0-standard-20080511.pdf) (PDF) from the original on 19 March 2012. Retrieved 4 July 2012. (Table 6.12)
26. *Physical layer standard for cdma2000 spread spectrum systems* (https://web.archive.org/web/20131116065606/http://www.3gpp2.org/public_html/specs/C.S0002-D_v2.0_051006.pdf) (PDF). Revision D version 2.0. 3rd Generation Partnership Project 2. October 2005. pp. 2–89–2–92. Archived from the original (http://www.3gpp2.org/public_html/specs/C.S0002-D_v2.0_051006.pdf) (PDF) on 16 November 2013. Retrieved 14 October 2013.
27. "11. Error correction strategy". *ETSI EN 300 751* (http://www.etsi.org/deliver/etsi_en/300700_300799/300751/01.02.01_60/en_300751v010201p.pdf) (PDF). V1.2.1. Sophia Antipolis, France: European Telecommunications Standards Institute. January 2003. pp. 67–8. Archived (https://web.archive.org/web/20151228050128/http://www.etsi.org/deliver/etsi_en/300700_300799/300751/01.02.01_60/en_300751v010201p.pdf) (PDF) from the original on 28 December 2015. Retrieved 26 January 2016.
28. "6 Bit CRC Zoo" (https://users.ece.cmu.edu/~koopman/crc/crc6.html). *users.ece.cmu.edu*. Archived (https://web.archive.org/web/20180407230334/http://users.ece.cmu.edu/~koopman/crc/crc6.html) from the original on 7 April 2018. Retrieved 19 January 2018.
29. Chakravarty, Tridib (December 2001). *Performance of Cyclic Redundancy Codes for Embedded Networks* (http://www.ece.cmu.edu/~koopman/thesis/chakravarty.pdf) (PDF) (Thesis). Philip Koopman, advisor. Carnegie Mellon University. pp. 5, 18. Archived (https://web.archive.org/web/20140101132228/http://www.ece.cmu.edu/~koopman/thesis/chakravarty.pdf) (PDF) from the original on 1 January 2014. Retrieved 8 July 2013.
30. "5.1.4 CRC-8 encoder (for packetized streams only)". *EN 302 307* (https://www.etsi.org/deliver/etsi_en/302300_302399/302307/01.03.01_60/en_302307v010301p.pdf) (PDF). V1.3.1. Sophia Antipolis, France: European Telecommunications Standards Institute. March 2013. p. 17. Archived (https://web.archive.org/web/20170830061535/http://www.etsi.org/deliver/etsi_en/302300_302399/302307/01.03.01_60/en_302307v010301p.pdf) (PDF) from the original on 30 August 2017. Retrieved 29 July 2016.
31. "8 Bit CRC Zoo" (https://users.ece.cmu.edu/~koopman/crc/crc8.html). *users.ece.cmu.edu*. Archived (https://web.archive.org/web/20180407224301/http://users.ece.cmu.edu/~koopman/crc/crc8.html) from the original on 7 April 2018. Retrieved 19 January 2018.

32. "7.2.1.2 8-bit 0x2F polynomial CRC Calculation". *Specification of CRC Routines* (https://web.archive.org/web/20160724123829/https://www.autosar.org/fileadmin/files/releases/4-2/software-architecture/safety-and-security/standard/AUTOSAR_SWS_CRCLibrary.pdf) (PDF). 4.2.2. Munich: AUTOSAR. 22 July 2015. p. 24. Archived from the original (https://www.autosar.org/fileadmin/files/releases/4-2/software-architecture/safety-and-security/standard/AUTOSAR_SWS_CRCLibrary.pdf) (PDF) on 24 July 2016. Retrieved 24 July 2016.
33. "5.1.1.8 Cyclic Redundancy Check field (CRC-8 / CRC-16)". *openSAFETY Safety Profile Specification: EPSG Working Draft Proposal 304* (https://web.archive.org/web/20170812202348/http://www.ethernet-powerlink.org/en/downloads/technical-documents/action/open-download/download/epsg-wdp-304-v-1-4-0/?no_cache=1). 1.4.0. Berlin: Ethernet POWERLINK Standardisation Group. 13 March 2013. p. 42. Archived from the original (http://www.ethernet-powerlink.org/en/downloads/technical-documents/action/open-download/download/epsg-wdp-304-v-1-4-0/?no_cache=1) on 12 August 2017. Retrieved 22 July 2016.
34. "B.7.1.1 HEC generation". *Specification of the Bluetooth System* (https://www.bluetooth.org/DocMan/handlers/DownloadDoc.ashx?doc_id=286439). Vol. 2. Bluetooth SIG. 2 December 2014. pp. 144–5. Archived (https://web.archive.org/web/20150326224908/https://www.bluetooth.org/DocMan/handlers/DownloadDoc.ashx?doc_id=286439) from the original on 26 March 2015. Retrieved 20 October 2014.
35. Whitfield, Harry (24 April 2001). "XFCNs for Cyclic Redundancy Check Calculations" (https://web.archive.org/web/20050525224339/http://homepages.cs.ncl.ac.uk/harry.whitfield/home.formal/CRCs.html). Archived from the original (http://homepages.cs.ncl.ac.uk/harry.whitfield/home.formal/CRCs.html) on 25 May 2005.
36. Richardson, Andrew (17 March 2005). *WCDMA Handbook* (https://books.google.com/books?id=yN5lve5L4vwC&pg=PA223). Cambridge University Press. p. 223. ISBN 978-0-521-82815-4.
37. *FlexRay Protocol Specification*. 3.0.1. Flexray Consortium. October 2010. p. 114. (4.2.8 Header CRC (11 bits))
38. Perez, A. (1983). "Byte-Wise CRC Calculations". *IEEE Micro*. **3** (3): 40–50. Bibcode:1983IMicr...3c..40P (https://ui.adsabs.harvard.edu/abs/1983IMicr...3c..40P). doi:10.1109/MM.1983.291120 (https://doi.org/10.1109%2FMM.1983.291120). S2CID 206471618 (https://api.semanticscholar.org/CorpusID:206471618).
39. Ramabadran, T.V.; Gaitonde, S.S. (1988). "A tutorial on CRC computations". *IEEE Micro*. **8** (4): 62–75. Bibcode:1988IMicr...8d..62R (https://ui.adsabs.harvard.edu/abs/1988IMicr...8d..62R). doi:10.1109/40.7773 (https://doi.org/10.1109%2F40.7773). S2CID 10216862 (https://api.semanticscholar.org/CorpusID:10216862).
40. "Longwave Radio Data Decoding using and HC11 and an MC3371" (https://web.archive.org/web/20150924015512/http://www.freescale.com/files/microcontrollers/doc/app_note/AN1597.pdf) (PDF). Freescale Semiconductor. 2004. AN1597/D. Archived from the original (http://www.freescale.com/files/microcontrollers/doc/app_note/AN1597.pdf) (PDF) on 24 September 2015.
41. Ely, S.R.; Wright, D.T. (March 1982). *L.F. Radio-Data: specification of BBC experimental transmissions 1982* (https://downloads.bbc.co.uk/rd/pubs/reports/1982-02.pdf) (PDF). Research Department, Engineering Division, The British Broadcasting Corporation. p. 9. Archived (https://web.archive.org/web/20131012064432/http://downloads.bbc.co.uk/rd/pubs/reports/1982-02.pdf) (PDF) from the original on 12 October 2013. Retrieved 11 October 2013.
42. *Cyclic Redundancy Check (CRC): PSoC Creator Component Datasheet* (http://www.cypress.com/file/128066/download). Cypress Semiconductor. 20 February 2013. p. 4. Archived (https://web.archive.org/web/20160202085601/http://www.cypress.com/file/128066/download) from the original on 2 February 2016. Retrieved 26 January 2016.

43. "Cyclic redundancy check (CRC) in CAN frames" (http://www.can-cia.org/can-knowledge/can/crc/). *CAN in Automation*. Archived (https://web.archive.org/web/20160201145928/http://www.can-cia.org/can-knowledge/can/crc/) from the original on 1 February 2016. Retrieved 26 January 2016.
44. "3.2.3 Encoding and error checking". *A signalling standard for trunked private land mobile radio systems (MPT 1327)* (http://www.ofcom.org.uk/static/archive/ra/publication/mpt/mpt_pdf/mpt1327.pdf) (PDF) (3rd ed.). Ofcom. June 1997. p. 3. Archived (https://web.archive.org/web/20120714015950/http://www.ofcom.org.uk/static/archive/ra/publication/mpt/mpt_pdf/mpt1327.pdf) (PDF) from the original on 14 July 2012. Retrieved 16 July 2012.
45. Rehmann, Albert; Mestre, José D. (February 1995). "Air Ground Data Link VHF Airline Communications and Reporting System (ACARS) Preliminary Test Report" (https://web.archive.org/web/20120802065800/http://ntl.bts.gov/lib/1000/1200/1290/tn95_66.pdf) (PDF). Federal Aviation Authority Technical Center. p. 5. Archived from the original (http://ntl.bts.gov/lib/1000/1200/1290/tn95_66.pdf) (PDF) on 2 August 2012. Retrieved 7 July 2012.
46. "6.2.5 Error control". *ETSI EN 300 175-3* (http://www.etsi.org/deliver/etsi_en/300100_300199/30017503/02.05.01_60/en_30017503v020501p.pdf) (PDF). V2.5.1. Sophia Antipolis, France: European Telecommunications Standards Institute. August 2013. pp. 99, 101. Archived (https://web.archive.org/web/20150701125736/http://www.etsi.org/deliver/etsi_en/300100_300199/30017503/02.05.01_60/en_30017503v020501p.pdf) (PDF) from the original on 1 July 2015. Retrieved 26 January 2016.
47. Thaler, Pat (28 August 2003). "16-bit CRC polynomial selection" (http://www.t10.org/ftp/t10/document.03/03-290r0.pdf) (PDF). INCITS T10. Archived (https://web.archive.org/web/20110728081616/http://www.t10.org/ftp/t10/document.03/03-290r0.pdf) (PDF) from the original on 28 July 2011. Retrieved 11 August 2009.
48. "8.8.4 Check Octet (FCS)". *PROFIBUS Specification Normative Parts* (https://web.archive.org/web/20081116195826/http://www.kuebler.com/PDFs/Feldbus_Multiturn/specification_DP.pdf) (PDF). 1.0. Vol. 9. Profibus International. March 1998. p. 906. Archived from the original (https://www.kuebler.com/PDFs/Feldbus_Multiturn/specification_DP.pdf) (PDF) on 16 November 2008. Retrieved 9 July 2016.
49. *CAN with Flexible Data-Rate Specification* (https://web.archive.org/web/20130822124728/http://www.bosch-semiconductors.de/media/pdf_1/canliteratur/can_fd_spec.pdf) (PDF). 1.0. Robert Bosch GmbH. 17 April 2012. p. 13. Archived from the original (http://www.bosch-semiconductors.de/media/pdf_1/canliteratur/can_fd_spec.pdf) (PDF) on 22 August 2013. (3.2.1 DATA FRAME)
50. "OS-9 Operating System System Programmer's Manual" (http://www.roug.org/soren/6809/os9sysprog.html#f.crc). *roug.org*. Archived (https://web.archive.org/web/20180717070700/http://www.roug.org/soren/6809/os9sysprog.html#f.crc) from the original on 17 July 2018. Retrieved 17 July 2018.
51. Koopman, Philip P. (20 May 2018). "24 Bit CRC Zoo" (https://users.ece.cmu.edu/~koopman/crc/crc8.html). *users.ece.cmu.edu*. Archived (https://web.archive.org/web/20180407224301/http://users.ece.cmu.edu/~koopman/crc/crc8.html) from the original on 7 April 2018. Retrieved 19 January 2018.
52. NVM Express (TM) Command Set Specification
53. "cksum" (http://pubs.opengroup.org/onlinepubs/9699919799/utilities/cksum.html). *pubs.opengroup.org*. Archived (https://web.archive.org/web/20180718084130/http://pubs.opengroup.org/onlinepubs/9699919799/utilities/cksum.html) from the original on 18 July 2018. Retrieved 27 June 2017.
54. Boutell, Thomas; Randers-Pehrson, Glenn; et al. (14 July 1998). "PNG (Portable Network Graphics) Specification, Version 1.2" (http://www.libpng.org/pub/png/spec/1.2/PNG-Structure.html). Libpng.org. Archived (https://web.archive.org/web/20110903114128/http://www.libpng.org/pub/png/spec/1.2/PNG-Structure.html) from the original on 3 September 2011. Retrieved 3 February 2011.

55. "ReFS integrity streams" (https://learn.microsoft.com/en-us/windows-server/storage/refs/integrity-streams).
56. "[MS-VHDX]: Structures" (https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-vhdx/340e64a4-ae2a-4dc1-b19b-3dd9d57a3359).
57. *AIXM Primer* (http://www.eurocontrol.int/sites/default/files/service/content/documents/information-management/20060320-aixm-primer.pdf) (PDF). 4.5. European Organisation for the Safety of Air Navigation. 20 March 2006. Archived (https://web.archive.org/web/20181120041622/https://www.eurocontrol.int/sites/default/files/service/content/documents/information-management/20060320-aixm-primer.pdf) (PDF) from the original on 20 November 2018. Retrieved 3 February 2019.
58. ETSI TS 100 909 (http://www.etsi.org/deliver/etsi_ts/100900_100999/100909/08.09.00_60/ts_100909v080900p.pdf) Archived (https://web.archive.org/web/20180417050648/http://www.etsi.org/deliver/etsi_ts/100900_100999/100909/08.09.00_60/ts_100909v080900p.pdf) 17 April 2018 at the Wayback Machine version 8.9.0 (January 2005), Section 4.1.2 a
59. Gammel, Berndt M. (31 October 2005). *Matpack documentation: Crypto – Codes* (http://www.matpack.de/index.html#DOWNLOAD). Matpack.de. Archived (https://web.archive.org/web/20130825205922/http://matpack.de/index.html#DOWNLOAD) from the original on 25 August 2013. Retrieved 21 April 2013. (Note: MpCRC.html is included with the Matpack compressed software source code, under /html/LibDoc/Crypto)
60. Geremia, Patrick (April 1999). "Cyclic redundancy check computation: an implementation using the TMS320C54x" (http://www.ti.com/lit/an/spra530/spra530.pdf) (PDF). Texas Instruments. p. 5. Archived (https://web.archive.org/web/20120614061303/http://www.ti.com/lit/an/spra530/spra530.pdf) (PDF) from the original on 14 June 2012. Retrieved 4 July 2012.
61. Jones, David T. "An Improved 64-bit Cyclic Redundancy Check for Protein Sequences" (http://www.cs.ucl.ac.uk/staff/d.jones/crcnote.pdf) (PDF). University College London. Archived (https://web.archive.org/web/20110607110311/http://www.cs.ucl.ac.uk/staff/d.jones/crcnote.pdf) (PDF) from the original on 7 June 2011. Retrieved 15 December 2009.

Further reading

- Warren Jr., Henry S. (2013). "14. Cyclic Redundancy Check" (https://books.google.com/books?id=VicPJYM0I5QC&pg=PA319). *Hacker's Delight* (2nd ed.). Addison Wesley. pp. 319–330. ISBN 978-0-321-84268-8.
- Koopman, Philip (2024). *Understanding Checksums and Cyclic Redundancy Checks*. ASIN B0CVXWDZ99 (https://www.amazon.com/dp/B0CVXWDZ99).

External links

- Mitra, Jubin; Nayak, Tapan (January 2017). "Reconfigurable very high throughput low latency VLSI (FPGA) design architecture of CRC 32". *Integration, the VLSI Journal*. **56**: 1–14. doi:10.1016/j.vlsi.2016.09.005 (https://doi.org/10.1016%2Fj.vlsi.2016.09.005).
- Cyclic Redundancy Checks (http://www.mathpages.com/home/kmath458.htm), MathPages, overview of error-detection of different polynomials
- Williams, Ross (1993). "A Painless Guide to CRC Error Detection Algorithms" (https://web.archive.org/web/20110903033652/http://www.ross.net/crc/crcpaper.html). Archived from the original (http://www.ross.net/crc/crcpaper.html) on 3 September 2011. Retrieved 15 August 2011.
- Black, Richard (1994). "Fast CRC32 in Software" (http://www.cl.cam.ac.uk/Research/SRG/bluebook/21/crc/crc.html). *The Blue Book*. Systems Research Group, Computer Laboratory, University of Cambridge. Algorithm 4 was used in Linux and Bzip2.

- Kounavis, M.; Berry, F. (2005). "A Systematic Approach to Building High Performance, Software-based, CRC generators" (http://www.intel.com/technology/comms/perfnet/download/CRC_generators.pdf) (PDF). Intel. Archived (https://web.archive.org/web/20061216135550/http://www.intel.com/technology/comms/perfnet/download/CRC_generators.pdf) (PDF) from the original on 16 December 2006. Retrieved 4 February 2007., Slicing-by-4 and slicing-by-8 algorithms
- Kowalk, W. (August 2006). "CRC Cyclic Redundancy Check Analysing and Correcting Errors" (http://einstein.informatik.uni-oldenburg.de/papers/CRC-BitfilterEng.pdf) (PDF). Universität Oldenburg. Archived (https://web.archive.org/web/20070611140619/http://einstein.informatik.uni-oldenburg.de/papers/CRC-BitfilterEng.pdf) (PDF) from the original on 11 June 2007. Retrieved 1 September 2006. — Bitfilters
- Warren, Henry S. Jr. "Cyclic Redundancy Check" (https://web.archive.org/web/20150503014404/http://www.hackersdelight.org/crc.pdf) (PDF). *Hacker's Delight*. Archived from the original (http://www.hackersdelight.org/crc.pdf) (PDF) on 3 May 2015. — theory, practice, hardware, and software with emphasis on CRC-32.
- Reverse-Engineering a CRC Algorithm (http://www.cosc.canterbury.ac.nz/greg.ewing/essays/CRC-Reverse-Engineering.html) Archived (https://web.archive.org/web/20110807100031/http://www.cosc.canterbury.ac.nz/greg.ewing/essays/CRC-Reverse-Engineering.html) 7 August 2011 at the Wayback Machine
- Cook, Greg. "Catalogue of parameterised CRC algorithms" (https://reveng.sourceforge.io/crc-catalogue/all.htm). *CRC RevEng*. Archived (https://web.archive.org/web/20200801122415/https://reveng.sourceforge.io/crc-catalogue/all.htm) from the original on 1 August 2020. Retrieved 18 September 2020.
- Koopman, Phil. "Blog: Checksum and CRC Central" (http://checksumcrc.blogspot.com/). — includes links to PDFs giving 16 and 32-bit CRC Hamming distances
  - — (April 2023). "Why Life Critical Networks Tend To Provide HD=6" (http://checksumcrc.blogspot.com/2023/04/why-life-critical-networks-tend-to.html).
- Koopman, Philip; Driscoll, Kevin; Hall, Brendan (March 2015). "Cyclic Redundancy Code and Checksum Algorithms to Ensure Critical Data Integrity" (http://www.tc.faa.gov/its/worldpac/techrpt/tc14-49.pdf) (PDF). Federal Aviation Administration. DOT/FAA/TC-14/49. Archived (https://web.archive.org/web/20150518073214/http://www.tc.faa.gov/its/worldpac/techrpt/tc14-49.pdf) (PDF) from the original on 18 May 2015. Retrieved 9 May 2015.
- Koopman, Philip (January 2023). *Mechanics of Cyclic Redundancy Check Calculations* (https://www.youtube.com/watch?v=qRqvdOAfxcA) – via YouTube.
- ISO/IEC 13239:2002: Information technology -- Telecommunications and information exchange between systems -- High-level data link control (HDLC) procedures (https://www.iso.org/standard/37010.html)
- CRC32-Castagnoli Linux Library (https://github.com/spotify/linux/blob/master/crypto/crc32c.c)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Cyclic_redundancy_check&oldid=1348510528"